

Workflow Pengembangan Project Reservasi Ruang

Timeline, aturan kerja 1 branch main, test Postman, dan starter code untuk 6 programmer

Target dokumen ini: pengerjaan rapi dan minim conflict walau hanya memakai branch main. Tidak ada workflow yang benar-benar 100% anti error. Karena itu, aturan di bawah dibuat agar error kecil, cepat ketahuan, dan cepat dibetulkan.

0. Prinsip Utama

- ? Pakai 1 branch saja: main.
- ? Setiap orang hanya mengedit file miliknya sesuai jobdesk.
- ? File bersama wajib diumumkan dulu sebelum diedit: SecurityConfig, application.properties, pom.xml, layout, README.
- ? Sebelum mulai kerja: Pull dulu lewat VS Code.
- ? Sebelum push: jalankan test, lalu Pull sekali lagi, baru Push.
- ? Commit kecil. Jangan menumpuk perubahan besar dalam satu push.
- ? Jangan auto-format seluruh project. Format hanya file yang sedang dikerjakan.
- ? Kalau conflict muncul, berhenti coding dulu. Selesaikan conflict sebelum lanjut.

1. Workflow GitHub Dengan VS Code GUI

Untuk pemilik repo

- ? Buka project di VS Code.
- ? Klik icon Source Control di sidebar kiri.
- ? Klik Publish Branch atau Publish to GitHub.
- ? Pilih repo private/public sesuai keputusan kelompok.
- ? Buka GitHub repo, masuk Settings > Collaborators, invite 5 anggota.
- ? Kirim link repo ke grup.

Untuk anggota

- ? Accept invite dari GitHub.
- ? Buka VS Code.
- ? Klik Source Control > Clone Repository.
- ? Paste URL repo GitHub.
- ? Pilih folder lokal, lalu Open.
- ? Jalankan aplikasi dan pastikan halaman login tampil.

Loop harian semua anggota

- ? Pull: Source Control > tiga titik > Pull.
- ? Edit file milik sendiri.
- ? Run test atau minimal run aplikasi.
- ? Stage Changes lewat tombol plus di Source Control.
- ? Isi commit message dengan format: [Nama] fitur singkat.
- ? Klik Commit.
- ? Pull lagi.
- ? Kalau aman, klik Sync Changes atau Push.

2. Kepemilikan File Agar Tidak Tabrakan

Anggota	Fokus	File utama
Amelia	User, Mahasiswa, auth, profil	entity/User, entity/Mahasiswa, auth, UserService, AuthService, UserRepository, User DTO, MahasiswaController, UserRestController, login, mahasiswa, users
Atha	Admin dan Notifiable	entity/Admin, entity/Notifiable, AdminRequest, AdminController, AdminRestController, template admin, bagian approval UI yang khusus admin

Anggota	Fokus	File utama
Tadzkiroh	Satpam dan akses ruangan	entity/Satpam, entity/AccessRecord, AccessControlService, AccessRecordRepository, Access DTO, SatpamController, AccessRestController, SatpamRestController, template satpam/access
Ali	Room dan reservasi teknis	entity/Room, RoomService, ReservationService, RoomRepository, ReservationRepository, Room DTO, ReservationRequest/Availability/Cancel, RoomRestController, ReservationRestController, template rooms/reservations form
'Aarif	Reservation, Approval, status	entity/Reservation, entity/Approval, ApprovalService, ApprovalRepository, ReservationStatus, ApprovalDecision, Approval DTO, ApprovalRestController, ReservationResponse, template approvals/reservations list
Ajda	Notification	entity/Notification, NotificationService, NotificationRepository, Notification DTO, NotificationRestController, template notifications

3. Aturan Postman Umum

- ? Buat Collection bernama Reservasi Ruang.
- ? Buat Environment dengan variable baseUrl = http://localhost:8080.
- ? Header default: Content-Type = application/json dan Accept = application/json.
- ? Tanggal pakai format YYYY-MM-DD, contoh 2026-06-10.
- ? Jam pakai format HH:mm, contoh 09:00.
- ? Endpoint di bawah adalah target API final yang dibuat bertahap. Saat ini project masih starter login, jadi endpoint fitur belum aktif sebelum anggota mengisinya.
- ? Untuk Postman, lebih mudah test endpoint /api/**. Form login MVC /login punya CSRF, jadi jangan dijadikan patokan utama untuk test API.

Kesepakatan response disarankan: sukses memakai 200 OK atau 201 Created berisi JSON data. Error validasi memakai 400. Data tidak ada memakai 404. Akses role salah memakai 403. Belum login memakai 401.

4. Timeline Kerja Bertahap

1. Ketua repo + semua anggota

Apa yang dilakukan: Push project ke GitHub, invite 5 anggota, semua accept invite, semua clone dari VS Code.

Kenapa ini dilakukan terlebih dahulu: Semua orang harus punya sumber project yang sama sebelum mulai coding. Ini mencegah file lokal berbeda-beda.

Yang diharapkan setelah selesai: Semua laptop punya project yang sama, branch aktif main, aplikasi bisa dibuka.

Test Postman:

Method + Endpoint	Body / Params	Hasil berhasil	Jika gagal
GET {{baseUrl}}/login	Tidak ada body.	200 OK dan HTML login tampil.	404: app belum jalan. 500: cek log terminal.

Starter code:

```
// Tidak ada starter code fitur.  
// Step ini hanya menyiapkan repo dan akses anggota.
```

Hasil step 1: Repo siap dipakai bersama dan semua anggota bisa pull/push.

2. Amelia

Apa yang dilakukan: Rapikan pondasi User, Mahasiswa, dan auth API starter. Jangan ubah fitur Room/Reservation/Admin/Satpam/Notification.

Kenapa ini dilakukan terlebih dahulu: Semua modul lain bergantung pada data user. Reservation butuh Mahasiswa. Approval butuh Admin. AccessRecord butuh Satpam.

Yang diharapkan setelah selesai: User domain jelas, login API target jelas, role tetap bisa dipakai.

Test Postman:

Method + Endpoint	Body / Params	Hasil berhasil	Jika gagal
POST {{baseUrl}}/api/auth/login	{"username":"mhs","password":"mhs"}	200 OK, role MAHASISWA, data user aman tanpa passwordHash.	401: username/password salah. 403: security belum disesuaikan untuk API.
GET {{baseUrl}}/api/users/me	Header login/session/token sesuai auth final.	200 OK, id/nama/email/noHp/role.	401: belum login. 500: data user belum sinkron.
PUT {{baseUrl}}/api/users/me	{"nama":"Budi","email":"budi@mail.com","noHp":"0812"}	200 OK, profil berubah.	400: format email/noHp salah.

Starter code:

```
@RequestMapping("/api/auth")  
// POST /login -> cek username/password, hasilkan user aman.  
  
@RequestMapping("/api/users")  
// GET /me -> ambil profil user login.  
// PUT /me -> ubah nama, email, noHp.  
  
// Jangan return passwordHash.
```

Hasil step 2: Fondasi user siap dipakai modul lain tanpa menunggu UI final.

3. Ali

Apa yang dilakukan: Buat Room domain dan API ruangan starter: daftar room, tambah room, ubah status aktif.

Kenapa ini dilakukan terlebih dahulu: Reservasi tidak bisa dibuat sebelum data ruangan jelas. Room adalah syarat utama validateAvailability.

Yang diharapkan setelah selesai: Room punya field roomId, namaRuang, gedung, kapasitas, statusAktif. Room nonaktif tidak bisa dipakai reservasi.

Test Postman:

Method + Endpoint	Body / Params	Hasil berhasil	Jika gagal
GET {{baseUrl}}/api/rooms	Tidak ada body.	200 OK, list room.	500: repository/service belum benar.
POST {{baseUrl}}/api/rooms	{"namaRuang":"Lab 1","gedung":"A","kapasitas":40,"statusAktif":true}	201 Created, roomId terisi.	400: kapasitas kosong/negatif.
PATCH {{baseUrl}}/api/rooms/{roomId}/status	{"statusAktif":false}	200 OK, statusAktif berubah.	404: roomId tidak ada.

Starter code:

```
@RequestMapping("/api/rooms")
// GET / -> list room.
// POST / -> tambah room.
// PATCH /{roomId}/status -> aktif/nonaktif.

// Field: roomId, namaRuang, gedung, kapasitas, statusAktif.
```

Hasil step 3: Data room siap dipakai oleh ReservationService.

4. Ali + koordinasi dengan Aarif

Apa yang dilakukan: Buat ReservationService starter: validasi waktu, cek availability, create reservation, cancel reservation.

Kenapa ini dilakukan terlebih dahulu: Setelah Room ada, barulah reservasi bisa diuji. Status awal dan aturan cancel harus disepakati dengan pemilik Approval.

Yang diharapkan setelah selesai: Mahasiswa bisa mengajukan reservasi. Jadwal bentrok ditolak. Reservasi bisa dibatalkan bila status mengizinkan.

Test Postman:

Method + Endpoint	Body / Params	Hasil berhasil	Jika gagal
POST {{baseUrl}}/api/reservations/availability	{"roomId":"R001","tanggal":"2026-06-10","jamMulai":"09:00","jamSelesai":"11:00"}	200 OK, {"available":true}.	400: jam salah. 404: room tidak ada.
POST {{baseUrl}}/api/reservations	{"mahasiswaId":"M001","roomId":"R001","tanggal":"2026-06-10","jamMulai":"09:00","jamSelesai":"11:00","tujuan":"Rapat"}	201 Created, status awal sesuai enum.	409: jadwal bentrok. 400: waktu tidak valid.
PATCH {{baseUrl}}/api/reservations/{reservationId}/cancel	{"alasanPembatalan":"Tidak jadi"}	200 OK, cancelledAt terisi.	400: status tidak boleh cancel.

Starter code:

```
@RequestMapping("/api/reservations")
// POST /availability -> cek jadwal.
// POST / -> ajukan reservasi.
// GET /my -> riwayat mahasiswa.
// PATCH /{reservationId}/cancel -> batalkan.

// Pakai Reservation.validasiWaktu() dan isCanBeCancelled().
```

Hasil step 4: Alur pengajuan reservasi dasar siap, belum masuk keputusan admin.

5. 'Aarif

Apa yang dilakukan: Buat Reservation status, Approval entity, ApprovalDecision, dan ApprovalService starter.

Kenapa ini dilakukan terlebih dahulu: Approval harus masuk setelah Reservation dasar ada. Approval mengubah status reservasi, jadi status harus rapi dulu.

Yang diharapkan setelah selesai: Admin bisa memberi keputusan setuju, tolak, atau revisi. reviewedAt dan catatan terisi.

Test Postman:

Method + Endpoint	Body / Params	Hasil berhasil	Jika gagal
GET {{baseUrl}}/api/approvals/pending	Tidak ada body.	200 OK, list reservasi menunggu approval.	403: bukan admin. 500: status enum belum sinkron.
POST {{baseUrl}}/api/approvals/{reservationId}/approve	{"adminId":"A001","catatan":"Disetujui"}	200 OK, keputusan APPROVED, status reservation berubah.	400: reservation tidak bisa disetujui.
POST {{baseUrl}}/api/approvals/{reservationId}/reject	{"adminId":"A001","catatan":"Bentrok agenda"}	200 OK, keputusan REJECTED.	400: catatan wajib bila reject.
POST {{baseUrl}}/api/approvals/{reservationId}/revision	{"adminId":"A001","catatan":"Lengkapi tujuan"}	200 OK, keputusan REVISION.	400: status tidak valid.

Starter code:

```
@RequestMapping("/api/approvals")
// GET /pending -> list butuh keputusan.
// POST /{reservationId}/approve -> setuju.
// POST /{reservationId}/reject -> tolak.
// POST /{reservationId}/revision -> minta revisi.

// Approval harus mengubah status Reservation.
```

Hasil step 5: Alur keputusan admin siap dipakai UI admin dan notifikasi.

6. Atha

Apa yang dilakukan: Buat Admin flow di controller/view: daftar reservasi pending, tombol setuju, tolak, minta revisi.

Kenapa ini dilakukan terlebih dahulu: Atha mengurus sisi Admin dan Notifiable. ApprovalService dari step 5 menjadi mesin prosesnya.

Yang diharapkan setelah selesai: Admin dapat melihat data yang perlu diproses dan mengirim keputusan dengan catatan.

Test Postman:

Method + Endpoint	Body / Params	Hasil berhasil	Jika gagal
GET {{baseUrl}}/api/admin/approvals/pending	Tidak ada body.	200 OK, list pending untuk admin.	404: endpoint belum diarahkan ke service approval.
POST {{baseUrl}}/api/admin/approvals/{reservationId}/decision	{"keputusan":"APPROVED","catatan":"OK"}	200 OK, ApprovalResponse.	400: keputusan tidak dikenal. 403: bukan admin.

Starter code:

```
@RequestMapping("/api/admin")
// GET /approvals/pending -> tampilkan antrian.
// POST /approvals/{reservationId}/decision -> kirim keputusan.

// Admin tetap memakai ApprovalService, jangan duplikasi logika.
```

Hasil step 6: Admin flow siap tanpa mengambil alih logic ApprovalService.

7. Tadzkirah

Apa yang dilakukan: Buat Satpam dan AccessRecord flow: check-in, check-out, lapor kendala.

Kenapa ini dilakukan terlebih dahulu: Access baru boleh dikerjakan setelah reservasi bisa disetujui. Satpam hanya memproses reservasi yang valid.

Yang diharapkan setelah selesai: Check-in membuat AccessRecord. Check-out melengkapi checkOutTime. Kendala tersimpan di catatanPelanggaran.

Test Postman:

Method + Endpoint	Body / Params	Hasil berhasil	Jika gagal
POST {{baseUrl}}/api/access/{{reservationId}}/check-in	{"satpamId":"S001"}	201 Created, checkInTime terisi.	400: reservation belum disetujui. 404: reservation tidak ada.
POST {{baseUrl}}/api/access/{{recordId}}/check-out	{"satpamId":"S001"}	200 OK, checkOutTime terisi.	400: belum check-in atau sudah check-out.
POST {{baseUrl}}/api/access/{{recordId}}/issue	{"catatanPelanggaran":"Kunci hilang"}	200 OK, catatan tersimpan.	404: recordId tidak ada.

Starter code:

```
@RequestMapping("/api/access")
// POST /{{reservationId}}/check-in -> buat AccessRecord.
// POST /{{recordId}}/check-out -> isi checkOutTime.
// POST /{{recordId}}/issue -> isi catatanPelanggaran.

// Hanya untuk reservation yang sudah disetujui.
```

Hasil step 7: Alur akses ruangan siap dipakai dashboard satpam.

8. Ajda

Apa yang dilakukan: Buat Notification flow: kirim notifikasi, lihat notifikasi, tandai dibaca, status update.

Kenapa ini dilakukan terlebih dahulu: Notification idealnya masuk setelah status Reservation/Approval/Access mulai berubah agar ada event yang dikirim.

Yang diharapkan setelah selesai: Setiap perubahan penting dapat mengirim pesan ke penerima yang tepat.

Test Postman:

Method + Endpoint	Body / Params	Hasil berhasil	Jika gagal
GET {{baseUrl}}/api/notifications	Tidak ada body.	200 OK, list notifikasi user login.	401: belum login. 500: penerima Notifiable belum sinkron.
POST {{baseUrl}}/api/notifications	{"penerimaId":"M001","penerimaRole":"MAHASISWA","pesan":"Reservasi disetujui"}	201 Created, statusBaca false.	400: penerima tidak valid.
PATCH {{baseUrl}}/api/notifications/{{notificationId}}/read	Tidak ada body.	200 OK, statusBaca true.	404: notificationId tidak ada.

Starter code:

```
@RequestMapping("/api/notifications")
// GET / -> list notifikasi penerima.
// POST / -> kirim notifikasi manual/system.
// PATCH /{{notificationId}}/read -> tandai dibaca.

// sendStatusUpdate dipanggil saat status reservation berubah.
```

Hasil step 8: Notifikasi siap dipakai semua modul tanpa mengubah logic modul lain.

9. Semua anggota

Apa yang dilakukan: Integrasi UI MVC setelah API/service inti stabil. Jangan mulai UI berat sebelum service bisa dites di Postman.

Kenapa ini dilakukan terlebih dahulu: UI yang dibuat sebelum service stabil biasanya sering dibongkar ulang. Postman membantu memastikan logic benar dulu.

Yang diharapkan setelah selesai: Halaman role menampilkan data nyata dari service masing-masing dan tetap sesuai hak akses.

Test Postman:

Method + Endpoint	Body / Params	Hasil berhasil	Jika gagal
GET {{baseUrl}}/mahasiswa/dashboard	Login sebagai mhs lewat browser.	200 OK, halaman mahasiswa.	403: role salah. 302: belum login.
GET {{baseUrl}}/admin/dashboard	Login sebagai adm lewat browser.	200 OK, halaman admin.	403: role salah. 302: belum login.
GET {{baseUrl}}/satpam/dashboard	Login sebagai stm lewat browser.	200 OK, halaman satpam.	403: role salah. 302: belum login.

Starter code:

```
// Controller MVC memanggil service yang sudah lolos Postman.  
// Template hanya menampilkan data, bukan tempat logic bisnis.  
// Jangan pindahkan logic service ke HTML.
```

Hasil step 9: UI mulai tersambung tanpa menabrak domain/service anggota lain.

10. Semua anggota dipimpin ketua repo

Apa yang dilakukan: Uji alur penuh: login, buat room, ajukan reservasi, approve, check-in, check-out, notifikasi.

Kenapa ini dilakukan terlebih dahulu: Ini memastikan semua modul saling nyambung dan tidak hanya benar sendiri-sendiri.

Yang diharapkan setelah selesai: Satu skenario end-to-end berjalan dari awal sampai selesai tanpa error.

Test Postman:

Method + Endpoint	Body / Params	Hasil berhasil	Jika gagal
1. POST /api/auth/login	{"username":"mhs","password":"mhs"}	Login mahasiswa sukses.	401: cek seed user.
2. POST /api/rooms	{"namaRuang":"Lab 1","gedung":"A","kapasitas":40,"statusAktif":true}	Room dibuat.	400: body salah.
3. POST /api/reservations	Body reservasi valid.	Reservation dibuat status awal.	409: jadwal bentrok.
4. POST /api/approvals/{id}/approve	Body admin/catatan.	Reservation disetujui.	403: bukan admin.
5. POST /api/access/{id}/check-in	{"satpamId":"S001"}	Check-in sukses.	400: belum approved.
6. POST /api/access/{recordId}/check-out	{"satpamId":"S001"}	Check-out sukses.	400: belum check-in.
7. GET /api/notifications	Tidak ada body.	Notifikasi status muncul.	500: Notifiable belum sinkron.

Starter code:

```
// Buat Postman Collection: End-to-End Reservasi.  
// Jalankan request berurutan.  
// Simpan id dari response manual ke request berikutnya.
```

Hasil step 10: Project siap masuk tahap polishing, validasi, dan presentasi.

5. Detail Body Postman Per Modul

Auth dan User

Method + Endpoint	Body / Params	Hasil berhasil	Jika gagal
POST {{baseUrl}}/api/auth/login	{"username":"mhs","password":"mhs"}	200 OK: userId, username, role.	401: akun salah. 403: konfigurasi security API belum siap.
GET {{baseUrl}}/api/users/me	Tidak ada body. Pakai login/session/token final.	200 OK: profil tanpa passwordHash.	401: belum login.
PUT {{baseUrl}}/api/users/me	{"nama":"Nama Baru","email":"nama@mail.com","noHp":"081234"}	200 OK: profil berubah.	400: validasi gagal.

Room

Method + Endpoint	Body / Params	Hasil berhasil	Jika gagal
GET {{baseUrl}}/api/rooms	Tidak ada body.	200 OK: list room.	500: service/repository error.
POST {{baseUrl}}/api/rooms	{"namaRuang":"Lab 1","gedung":"A","kapasitas":40,"statusAktif":true}	201 Created: RoomResponse.	400: field wajib kosong.
PATCH {{baseUrl}}/api/rooms/R001/status	{"statusAktif":false}	200 OK: status berubah.	404: room tidak ditemukan.

Reservation

Method + Endpoint	Body / Params	Hasil berhasil	Jika gagal
POST {{baseUrl}}/api/reservations/availability	{"roomId":"R001","tanggal":"2026-06-10","jamMulai":"09:00","jamSelesai":"11:00"}	200 OK: available true/false.	400: jam tidak valid.
POST {{baseUrl}}/api/reservations	{"mahasiswaId":"M001","roomId":"R001","tanggal":"2026-06-10","jamMulai":"09:00","jamSelesai":"11:00","tujuan":"Rapat"}	201 Created: ReservationResponse.	409: bentrok. 404: room/mahasiswa tidak ada.
GET {{baseUrl}}/api/reservations/my	Tidak ada body.	200 OK: riwayat mahasiswa.	401: belum login.
PATCH {{baseUrl}}/api/reservations/RSV001/cancel	{"alasanPembatalan":"Tidak jadi"}	200 OK: cancelledAt terisi.	400: tidak bisa cancel.

Approval dan Admin

Method + Endpoint	Body / Params	Hasil berhasil	Jika gagal
GET {{baseUrl}}/api/approvals/pending	Tidak ada body.	200 OK: list pending.	403: bukan admin.
POST {{baseUrl}}/api/approvals/RSV001/approve	{"adminId":"A001","catatan":"Disetujui"}	200 OK: keputusan approved.	400: status tidak valid.
POST {{baseUrl}}/api/approvals/RSV001/reject	{"adminId":"A001","catatan":"Ditolak karena jadwal"}	200 OK: keputusan rejected.	400: catatan kosong.
POST {{baseUrl}}/api/approvals/RSV001/revision	{"adminId":"A001","catatan":"Lengkapi tujuan"}	200 OK: keputusan revision.	400: status tidak valid.

Access dan Satpam

Method + Endpoint	Body / Params	Hasil berhasil	Jika gagal
POST {{baseUrl}}/api/access/RSV001/check-in	{"satpamId":"S001"}	201 Created: AccessRecord.	400: reservation belum approved.
POST {{baseUrl}}/api/access/ACC001/check-out	{"satpamId":"S001"}	200 OK: checkOutTime terisi.	400: belum check-in/sudah checkout.
POST {{baseUrl}}/api/access/ACC001/issue	{"catatanPelanggaran":"Pintu rusak"}	200 OK: catatan tersimpan.	404: access record tidak ada.

Notification

Method + Endpoint	Body / Params	Hasil berhasil	Jika gagal
GET {{baseUrl}}/api/notifications	Tidak ada body.	200 OK: list notifikasi.	401: belum login.
POST {{baseUrl}}/api/notifications	{"penerimaId":"M001","penerimaRole":"MAHASISWA","pesan":"Reservasi disetujui"}	201 Created: statusBaca false.	400: penerima tidak valid.
PATCH {{baseUrl}}/api/notifications/NTF001/read	Tidak ada body.	200 OK: statusBaca true.	404: notification tidak ada.

6. Starter Code Ringkas Per Programmer

Starter code di bawah hanya contoh bentuk file. Isi logic tetap dikerjakan anggota masing-masing. Jangan copy sebagai implementasi selesai.

Amelia - User/Auth/Mahasiswa

```
// UserRestController
@RequestMapping("/api/users")
// GET /me
// PUT /me

// AuthRestController bila dipakai
@RequestMapping("/api/auth")
// POST /login

// Entity target: User, Mahasiswa
// Field: id, nama, email, noHp, passwordHash, nim, prodi, angkatan.
```

Atha - Admin/Notifiable

```
// AdminRestController
@RequestMapping("/api/admin")
// GET /approvals/pending
// POST /approvals/{reservationId}/decision

// Entity target: Admin extends User, Notifiable
// Field: unitKerja.
// Method: verifikasiReservasi, setujuReservasi, tolakReservasi, mintaRevisiData.
```

Tadzkiroh - Satpam/AccessRecord

```
// AccessRestController
@RequestMapping("/api/access")
// POST /{reservationId}/check-in
// POST /{recordId}/check-out
// POST /{recordId}/issue

// Entity target: Satpam, AccessRecord
// Field: shift, posJaga, checkInTime, checkOutTime, catatanPelanggaran.
```

Ali - Room/ReservationService

```
// RoomRestController
@RequestMapping("/api/rooms")
// GET /
// POST /
// PATCH /{roomId}/status

// ReservationRestController
@RequestMapping("/api/reservations")
// POST /availability
// POST /
// GET /my
// PATCH /{reservationId}/cancel
```

'Aarif - Reservation/Approval

```
// ApprovalRestController
@RequestMapping("/api/approvals")
// GET /pending
// POST /{reservationId}/approve
// POST /{reservationId}/reject
// POST /{reservationId}/revision

// Entity target: Reservation, Approval
// Enum target: ReservationStatus, ApprovalDecision.
```

Ajda - Notification

```
// NotificationRestController
@RequestMapping("/api/notifications")
// GET /
// POST /
// PATCH /{notificationId}/read

// Entity target: Notification
// Field: notificationId, penerima, pesan, statusBaca, createdAt.
```

7. Skenario Kendala dan Cara Menanganinya

Kendala	Penyebab umum	Penanganan	Pencegahan
Push ditolak	Ada commit baru di GitHub.	Jangan ulang push. Klik Pull di VS Code, selesaikan conflict jika ada, test, lalu Push lagi.	Selalu Pull sebelum kerja dan sebelum Push.
Conflict saat Pull	Dua orang edit baris/file yang sama.	Buka Merge Editor VS Code. Hubungi pemilik file. Pilih perubahan yang benar. Jangan asal Accept All.	Satu file satu owner. File bersama harus diumumkan dulu.
Aplikasi tidak jalan	MySQL mati, port 8080 dipakai, dependency belum siap, atau config salah.	Cek terminal. Jalankan MySQL. Ganti port jika perlu. Jalankan Maven reload.	Sebelum push, minimal run aplikasi lokal.
Database tidak muncul	Aplikasi belum jalan atau URL/kredensial salah.	Jalankan app. Cek application.properties. Refresh phpMyAdmin. Pastikan user root/password sesuai.	Jangan ubah config database tanpa kabar grup.
Endpoint 404	Mapping controller belum dibuat atau path salah.	Cek @RequestMapping dan URL Postman. Samakan path dengan dokumen ini.	Tulis endpoint di komentar sebelum implementasi.
Endpoint 400	Body JSON salah atau validasi gagal.	Cek Content-Type, nama field, format tanggal/jam.	Pakai contoh body Postman dari dokumen ini.
Endpoint 401/403	Belum login, role salah, atau security belum sesuai API.	Login dengan role benar. Cek SecurityConfig. Jangan buka endpoint role lain.	Uji tiap endpoint memakai user role yang sesuai.
Test gagal setelah Pull	Commit terakhir merusak alur.	Stop fitur baru. Baca error. Hubungi pemilik commit terakhir. Fix dulu sebelum lanjut.	Jalankan test sebelum Push. Commit kecil.
Anggota tidak hadir	File owner belum selesai.	Jangan ambil file tanpa izin. Ketua tentukan pengganti sementara dan catat di grup.	Setiap anggota push progres kecil setiap selesai subtask.
Logic duplikat	Controller atau HTML menaruh logic service.	Pindahkan logic ke service milik owner. Controller hanya panggil service.	Sepakati: entity data, service logic, controller jalan masuk, template tampilan.

8. Checklist Sebelum Setiap Push

- ? Sudah Pull terbaru dari main.
- ? Hanya edit file sesuai jobdesk.
- ? Tidak mengubah file shared tanpa kabar grup.
- ? Aplikasi bisa run atau test terkait lulus.
- ? Postman request modul sendiri sudah dicoba.
- ? Commit message jelas: [Nama] isi perubahan.
- ? Pull lagi sebelum Push.
- ? Kalau conflict, selesaikan dulu, jangan lanjut fitur baru.

9. Urutan Paling Aman

Urutan paling aman adalah: repo siap, User/Auth, Room, Reservation, Approval/Admin, Access/Satpam, Notification, UI, integrasi penuh. Urutan ini mengikuti ketergantungan data di class diagram. Jangan mulai fitur yang bergantung pada fitur lain sebelum API dasarnya lulus Postman.